



# Entornos de Desarrollo

*UT1- Desarrollo de software y  
lenguajes de programación*



# INTRODUCCIÓN

## EN ESTA UNIDAD APRENDEREMOS A:

- Reconocer la relación de los programas con los componentes del sistema informático
- Identificar las fases de desarrollo de un programa informático
- Clasificar los lenguajes de programación



# TIPOS DE SOFTWARE

- **DE SISTEMA** (Sistema operativo, drivers)
- **DE APLICACIÓN** (Suite ofimática, navegador, edición de imagen,...)
- **DE DESARROLLO** (Editores, compiladores, intérpretes,...)



# RELACIÓN HARDWARE - SOFTWARE

- **Disco duro** : Almacena de forma permanente los archivos ejecutables y los archivos de datos.
- **Memoria RAM** : Almacena de forma temporal el código binario de los archivos ejecutables y los archivos de datos necesarios.
- **CPU** : Lee y ejecuta instrucciones almacenadas en memoria RAM, así como los datos necesarios.
- **E/S** : Recoge nuevos datos desde la entrada, se muestran los resultados, se leen/guardan a disco, ...



# **CICLO DE VIDA DEL SOFTWARE**



# INGENIERÍA DEL SOFTWARE

*Disciplina que estudia los principios y metodologías para el desarrollo y mantenimiento de sistemas software.*



# DESARROLLO DE SOFTWARE

## FASES PRINCIPALES:

- *PLANIFICACIÓN*
- *ANÁLISIS*
- *DISEÑO*
- *CODIFICACIÓN*
- *PRUEBAS*
- *MANTENIMIENTO*



# ANÁLISIS

- Se determina y define claramente las necesidades del cliente y se especifica los requisitos que debe cumplir el software a desarrollar.
- La especificación de requisitos debe:
  - Ser completa y sin omisiones
  - Ser concisa y sin trivialidades
  - Evitar ambigüedades
  - Evitar detalles de diseño o implementación
  - Ser entendible por el cliente
  - Separar requisitos funcionales y no funcionales
  - Dividir y jerarquizar el modelo (diagramas de clases y casos de uso)
  - Fijar criterios de validación





# DISEÑO

- Se descompone y organiza el sistema en elementos componentes que pueden ser desarrollados por separado (generalmente se parte de los casos de uso)
- Se especifica la interrelación y funcionalidad de los elementos componentes (hay que describirlos paso a paso)
- Las actividades habituales son las siguientes:
  - Diseño detallado
  - Diseño de datos
  - Diseño de interfaz



# CODIFICACIÓN (implementación-programación)

- Se selecciona el entorno de desarrollo y el lenguaje
- Se escribe el código fuente de cada componente
- Pueden utilizarse distintos lenguajes informáticos:
  - **Lenguajes de programación** : C, C++, Java, Javascript, Python,...
  - **Lenguajes de otro tipo** : HTML, XML, JSON,...



# CODIFICACIÓN (implementación-programación)

- Es importante definir normas para que los programas tengan un aspecto homogéneo:
  - Comentarios
  - Declaraciones de variables
  - Nombres de clases, atributos, métodos...
  - Líneas en blanco
  - Sangrados



# PRUEBAS

- El principal objetivo de las pruebas debe ser conseguir que el programa funcione incorrectamente y que se descubran defectos
- Deberemos someter al programa al máximo número de situaciones diferentes (validando distintos caminos internamente y resultados externos)

Podemos comenzar por los componentes más pequeños hasta alcanzar la aplicación/programa completo



# MANTENIMIENTO

- Debemos someter al programa al máximo número de situaciones diferentes.
- Para ello hay que rehacer parte del trabajo realizado en las fases previas.
- Tipos de mantenimiento:
  - **Correctivo**: se corrigen defectos
  - **Perfectivo**: se mejora la funcionalidad
  - **Evolutivo**: se añade funcionalidades nuevas
  - **Adaptativo**: se adapta a nuevos entornos



# RESULTADOS TRAS CADA FASE (I)

- **ANÁLISIS**: Especificación de requisitos del software
- **DISEÑO detallado**: Especificación de módulos y funciones
- **CODIFICACIÓN**: Código fuente



# RESULTADOS TRAS CADA FASE (II)

- **PRUEBAS de unidades:** Módulos utilizables
- **PRUEBAS de integración:** Sistema utilizable
- **PRUEBAS del sistema:** Sistema aceptado
- **Documentación:** Documentación técnica y de usuario
- **MANTENIMIENTO:** Informes de errores y control de cambios



# ROLES EN EL DESARROLLO DE SOFTWARE

Para llevar a cabo el desarrollo de software es habitual que se integre un equipo con diferentes personas expertas en cada una de las áreas. Un ejemplo de los roles o “papeles” que se pueden integrar sería:

- Jefes de proyecto
- Expertos en el área del problema
- Analistas
- Arquitectos
- Diseñadores
- Programadores
- Probadores
- Implementadores





# METODOLOGÍAS DE DESARROLLO DE SOFTWARE

Las fases explicadas anteriormente se pueden organizar de diferentes maneras dependiendo del proyecto, el personal involucrado, del tiempo disponible, etc.

Al utilizar una metodología de desarrollo de software adecuada se facilita el trabajo, se agiliza, y se puede organizar de manera de cumplir con las fechas previstas.

Se diferencian dos grandes grupos de metodologías de desarrollo de software en la actualidad:

- Las **tradicionales/pesadas**

- Las **ágiles**



# METODOLOGÍAS DE DESARROLLO DE SOFTWARE

*Son un conjunto de procedimientos, técnicas y ayudas a la documentación para el desarrollo de productos software, en el que se van indicando paso a paso todas las actividades a realizar para lograr el producto informático deseado*

*En alguna bibliografía se diferencias los **modelos** (solo como las actividades que se deben llevar a cabo) de las **metodologías** (según la definición anterior)*



# MODELOS DE DESARROLLO DE SOFTWARE

- **Modelos clásicos (predictivos)**
  - Modelo en cascada
- **Modelos incrementales**
  - Modelo en espiral (iterativos)
- **Modelos evolutivos**
  - Construcción de Prototipos

# MODELO EN CASCADA (I)



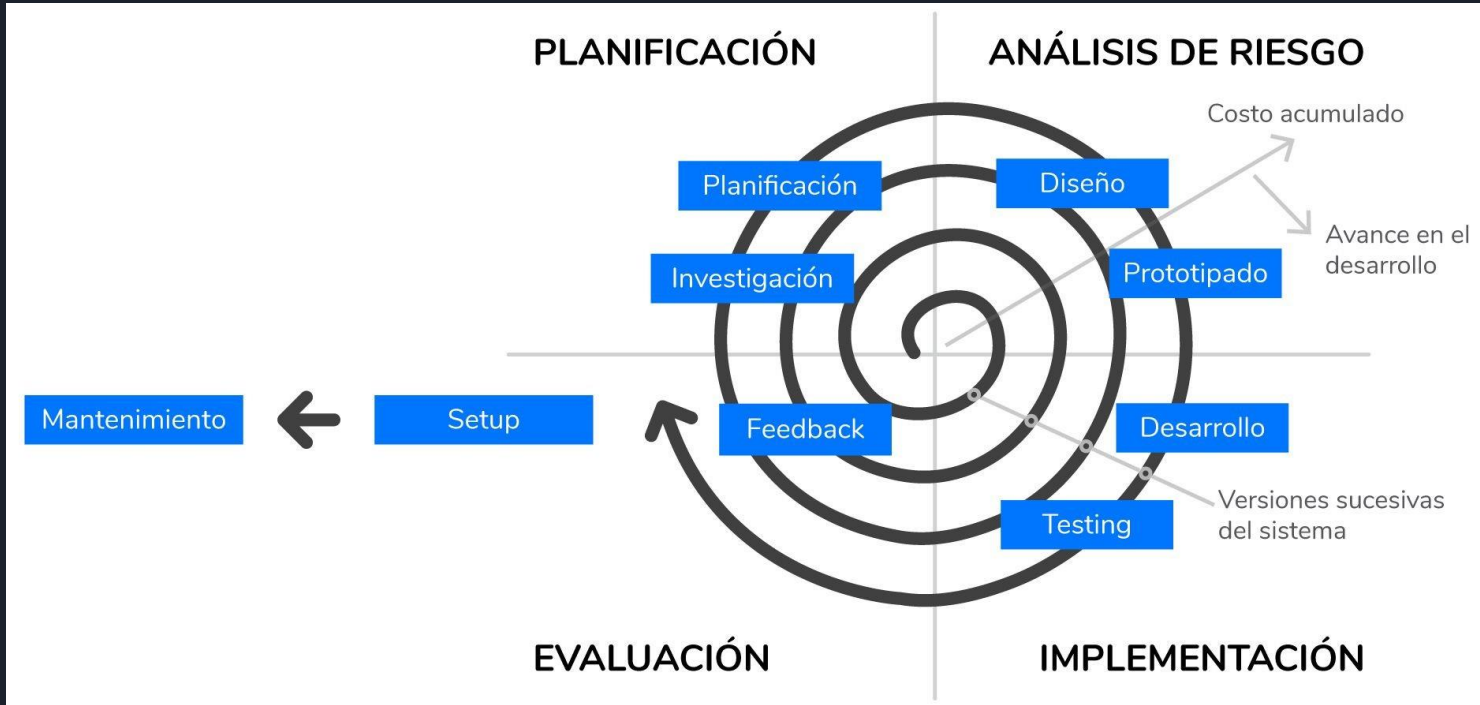


# MODELO EN CASCADA (II)

- Modelo de mayor antigüedad (clásico)
- Identifica las fases principales del desarrollo software
- Las fases han de realizarse en el orden indicado (secuencial)
- El resultado de una fase es la entrada de la siguiente fase
- Es un modelo bastante rígido que se adapta mal al cambio continuo de especificaciones (aunque permite volver atrás si es necesario)
- Existen diferentes variantes con mayor o menor cantidad de actividades

# MODELO EN ESPIRAL

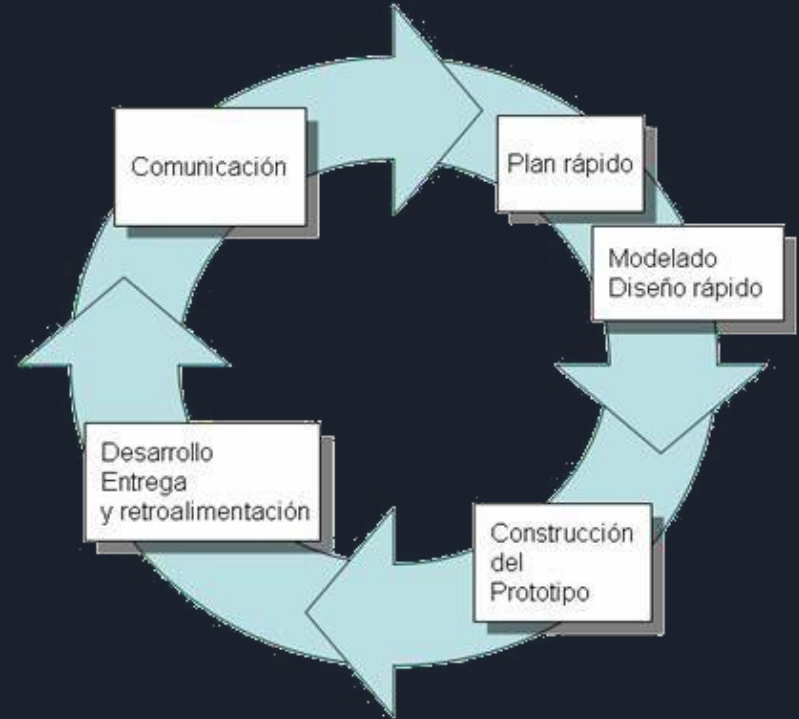
- Desarrollado por Boehm en 1988.



# PROTOTIPOS (I)

En estos modelos, a menudo los requisitos NO están especificados claramente:

- Por no existir experiencia previa
- Por omisión o falta de concreción del usuario/cliente





# PROTOTIPOS (II)

Proceso:

- Se crea un prototipo durante la fase de análisis y es probado por el usuario/cliente para refinar los requisitos del software a desarrollar
- Se repite el paso anterior las veces necesarias





# PROTOTIPOS (III)

Tipos de prototipos:

- **Prototipos rápidos**

- El prototipo puede estar desarrollado usando otro lenguaje y/o herramientas
- Finalmente el prototipo se desecha

- **Prototipos evolutivos**

- El prototipo está diseñado en el mismo lenguaje y herramientas del proyecto
- El prototipo se usa como base para desarrollar el proyecto



# METODOLOGÍAS ÁGILES (I)

- Son métodos de ingeniería del software basados en el desarrollo iterativo e incremental
- Los requisitos y soluciones evolucionan con el tiempo según la necesidad del proyecto
- El trabajo es realizado mediante la colaboración de equipos auto-organizados y multidisciplinares, inmersos en un proceso compartido de toma de decisiones a corto plazo
- Las metodologías más conocidas son:
  - **Scrum**
  - **XP** (eXtreme Programming)



# METODOLOGÍAS ÁGILES (II)

## Manifiesto por el Desarrollo Ágil

Estamos descubriendo formas mejores de desarrollar software tanto por nuestra propia experiencia como ayudando a terceros. A través de este trabajo hemos aprendido a valorar:

**Individuos e interacciones** sobre procesos y herramientas

**Software funcionando** sobre documentación extensiva

**Colaboración con el cliente** sobre negociación contractual

**Respuesta ante el cambio** sobre seguir un plan

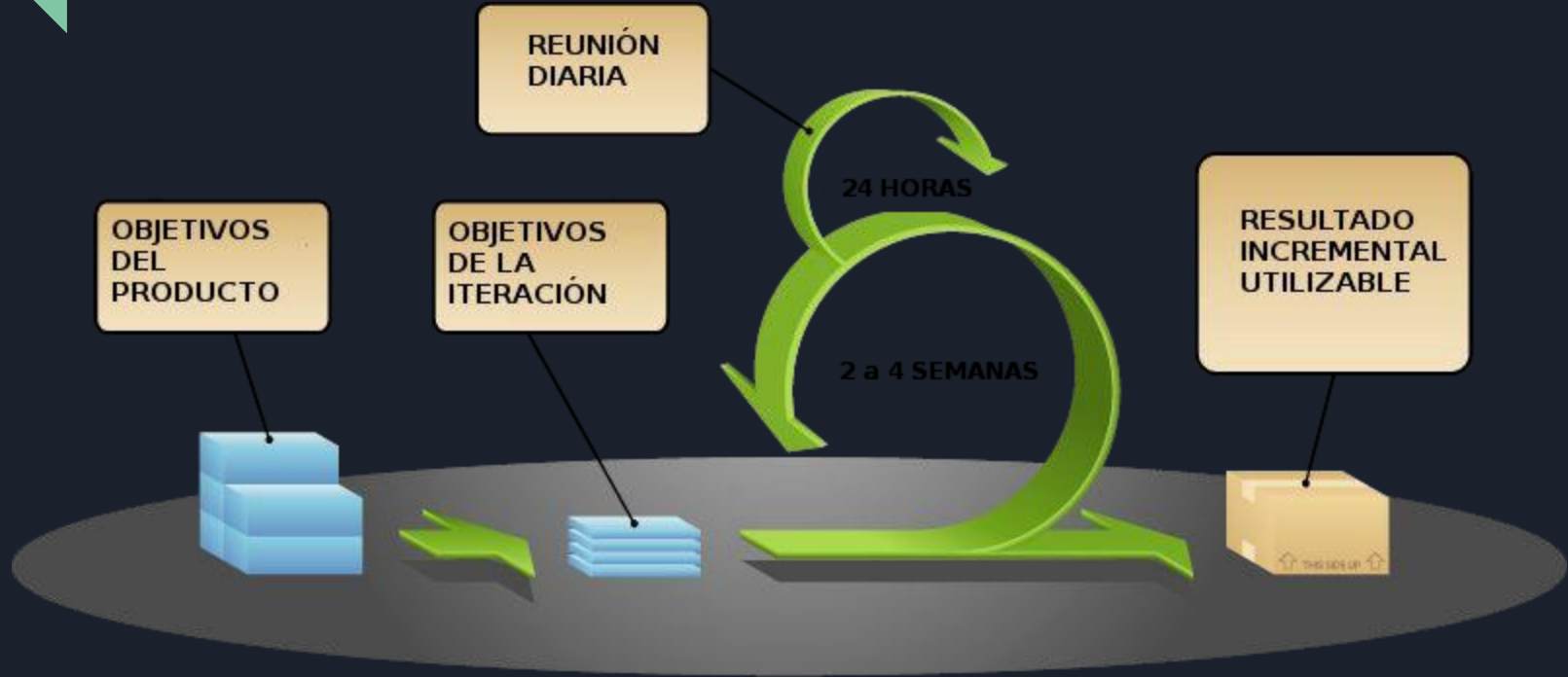
Esto es, aunque valoramos los elementos de la derecha, valoramos más los de la izquierda



# SCRUM (I)

- Modelo de desarrollo incremental
- Iteraciones (sprint) regulares cada 2 a 4 semanas
- Al principio de cada iteración se establecen sus objetivos priorizados (sprint backlog)
- Al finalizar cada iteración se obtiene una entrega parcial utilizable por el cliente
- Existen reuniones diarias para tratar la marcha del sprint

# SCRUM (II)





# XP (PROGRAMACIÓN EXTREMA)

## Características:

- Diseño sencillo
- Pequeñas mejoras continuas
- Pruebas y refactorización
- Integración continua
- Programación por parejas
- El cliente se integra en el equipo de desarrollo
- Estándares de codificación
- 40 horas semanales



# LENGUAJES DE PROGRAMACIÓN



# OBTENCIÓN DE CÓDIGO EJECUTABLE

Para obtener código binario ejecutable tenemos 2 opciones:

- **Compilar:**

- Traduce todo el código antes de ejecutarlo

- **Interpretar:**

- Procesa el código durante la ejecución, línea a línea
- Lee, analiza y prepara cada instrucción para el procesador
- Se detectan de inmediato las líneas de código problemáticas





# LENGUAJES COMPILADOS

## LENGUAJES COMPILADOS

- Ejemplos: C, C++
- Principal ventaja: Ejecución muy eficiente
- Principal desventaja: Es necesario compilar cada vez que el código fuente es modificado



# LENGUAJES INTERPRETADOS

- Ejemplos: PHP, Javascript
- Principal ventaja: El código fuente se interpreta directamente
- Principal desventaja: Ejecución menos eficiente



# PROCESO DE COMPILACIÓN/INTERPRETACIÓN

|                                            | Intérprete                                  | Compilador                                  |
|--------------------------------------------|---------------------------------------------|---------------------------------------------|
| Momento en que se traduce el código fuente | Durante el tiempo de ejecución del software | Antes de ejecutar el software               |
| Procedimiento de traducción                | Línea por línea                             | Siempre todo el código                      |
| Presentación de errores de código          | Después de cada línea                       | En conjunto, después de toda la compilación |
| Velocidad de traducción                    | Alta                                        | Baja                                        |
| Eficiencia de traducción                   | Baja                                        | Alta                                        |
| Coste de desarrollo                        | Bajo                                        | Alto                                        |
| Lenguajes típicos                          | PHP, Perl, Python, Ruby, BASIC              | C, C++, Pascal                              |



# TIPOS DE LENGUAJES

Según la forma en la que operan:

- **Declarativos:** indicamos el resultado a obtener sin especificar los pasos
- **Imperativos:** indicamos los pasos a seguir para obtener un resultado

# TIPOS DE LENGUAJES

Esta sería una implementación imperativa de un reductor:

```
1.  const arr = [2, 4, 6]
2.  let acc = 0
3.
4.  for (let i = 0; i < arr.length; i++){
5.    acc += arr[i]
6.  }
7.
8.  console.log(`El sumatorio resulta en ${acc}`)
9.  // El sumatorio resulta en 12
```

Frente a su equivalencia declarativa mediante una función personalizada `arrayReduce()` :

```
1.  const arr = [2, 4, 6]
2.  const arraySum = arrayReducer(arr)
3.
4.  console.log(`El sumatorio resulta en ${arraySum}`)
5.  // El sumatorio resulta en 12
```



# TIPOS DE LENGUAJES

Tipos de lenguajes declarativos:

- **Lógicos**: Utilizan reglas. Ej: Prolog
- **Funcionales**: Utilizan funciones. Ej: Lisp, Haskell
- **Algebraicos**: Utilizan sentencias. Ej: SQL

Normalmente son lenguajes interpretados



# TIPOS DE LENGUAJES

Tipos de lenguajes imperativos:

- **Estructurados:** C
- **Orientados a objetos:** C++, Java

Los lenguajes orientados a objetos son también lenguajes estructurados

Muchos de estos lenguajes son compilados



# TIPOS DE LENGUAJES (IV)

Tipos de lenguajes según nivel de abstracción:

- **Bajo nivel:** ensamblador
- **Medio nivel:** C
- **Alto nivel:** C++, Java





# EVOLUCIÓN DE LOS LENGUAJES DE PROGRAMACIÓN

- Código binario
- Ensamblador
- Lenguajes estructurados
- Lenguajes orientados a objetos



# CRITERIOS PARA LA SELECCIÓN DE UN LENGUAJE

- Campo de aplicación
- Experiencia previa
- Herramientas de desarrollo
- Documentación disponible
- Base de usuarios
- Reusabilidad
- Portabilidad
- Imposición del cliente